

Network and Cloud Infrastructures Project Work - SDN

Network Slicing with Ryu and Mininet

Davide Di Matteo - M63001865
dav.dimatteo@studenti.unina.it

Università degli studi di Napoli Federico II — 27/03/2026

Introduzione

The project implements network slicing in a virtualised SDN environment deployed on an **Ubuntu 22.04 LTS virtual machine**. The experimental setup is based on:

- **Mininet**, used to emulate hosts, switches, and links;
- **Open vSwitch** (via Mininet), used as the software data plane;
- **Python 3** scripts, used to define topologies and automate tests;
- **Ryu controller**, used to implement SDN control logic and OpenFlow rules.

The implementation is organised into the following sections:

1. **Topology-slicing**: restriction of communication paths between specific hosts;
2. **Service-slicing**: differentiated treatment of selected traffic classes.

For each section, validation tests are performed (e.g., connectivity and traffic checks) to evaluate the effectiveness of the proposed slicing policies.

1 Topology slicing

This section covers the definition of the network topology in Mininet and the resulting division into two isolated *slices*. The aim is to construct a topology in which hosts can communicate via predefined routes:

- The host h1 can communicate only with h3 via the **upper path**;
- The host h2 can communicate only with h4 via the **lower path**.

A this purpose, the following Mininet configuration has been defined by implementing the constructor of the `Mininet.Environment` class, whose logical view is represented in figure 1.

```
1 class Environment(object):
2     def __init__(self):
3         # ...
4         # Nodes definition
5         # Hosts definition
```

```

6 self.h1 = self.net.addHost('h1', mac= '00:00:00:00:00:01', ip= '10.0.0.1')
7 self.h2 = self.net.addHost('h2', mac= '00:00:00:00:00:02', ip= '10.0.0.2')
8 self.h3 = self.net.addHost('h3', mac= '00:00:00:00:00:03', ip= '10.0.0.3')
9 self.h4 = self.net.addHost('h4', mac= '00:00:00:00:00:04', ip= '10.0.0.4')
10 # Switches definition
11 self.s1 = self.net.addSwitch('s1')
12 self.s2 = self.net.addSwitch('s2')
13 self.Up = self.net.addSwitch('s3') # up
14 self.Dw = self.net.addSwitch('s4') # down
15
16 # Links definition
17 # Hosts to Switches
18 self.net.addLink(self.h1, self.s1, delay='0.01ms', port1=1, port2=1)
19 self.net.addLink(self.h2, self.s1, delay='0.01ms', port1=1, port2=2)
20 self.net.addLink(self.h3, self.s2, delay='0.01ms', port1=1, port2=3)
21 self.net.addLink(self.h4, self.s2, delay='0.01ms', port1=1, port2=4)
22 # Switches to Switches
23 self.net.addLink(self.s1, self.Up, bw=10, delay='0.025ms', port1=3, port2=1)
24 self.net.addLink(self.Up, self.s2, bw=10, delay='0.025ms', port1=2, port2=1)
25 self.net.addLink(self.s1, self.Dw, bw=1, delay='0.025ms', port1=4, port2=1)
26 self.net.addLink(self.Dw, self.s2, bw=1, delay='0.025ms', port1=2, port2=2)
27 # ...

```

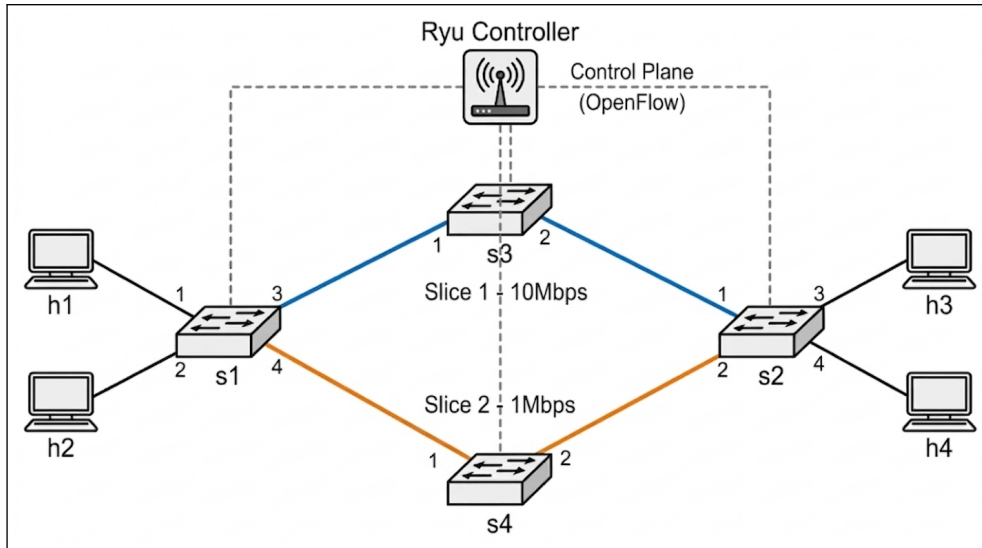


Figure 1: Logical view of the network topology

Given the defined network topology, a **deterministic** routing approach was implemented within the controller. This design ensures granular control over traffic flows, allowing for the enforcement of static routes that guarantee slice isolation and adherence to predefined network policies.

RyuController is an SDN application based on the OpenFlow 1.3 protocol, designed to manage a network environment emulated via Mininet and Open vSwitch. The application was developed by extending the `ryu.base.app_manager.RyuApp` base class. To ensure deterministic forwarding and strict slice isolation, a static addressing scheme was adopted: host MAC addresses are defined as constant strings directly within the class constructor."

```

1 # Base class extension
2 class RyuController(app_manager.RyuApp):
3     # OpenFlow 1.3
4     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
5
6     def __init__(self, *args, **kwargs):
7         super(RyuController, self).__init__(*args, **kwargs)
8
9         # Static MAC addresses definition for deterministic routing
10        self.h1 = '00:00:00:00:00:01'
11        self.h2 = '00:00:00:00:00:02'
12        self.h3 = '00:00:00:00:00:03'
13        self.h4 = '00:00:00:00:00:04'

```

```

14
15     # Utility function to add flow rules to the switches
16     def add_flow(self, datapath, priority, match, actions, idle_timeout=0, flag=0):
17         ofproto = datapath.ofproto
18         parser = datapath.ofproto_parser
19
20         inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]
21         mod = parser.OFPFlowMod(
22             datapath=datapath, priority=priority,
23             match=match, instructions=inst,
24             idle_timeout = idle_timeout, flags=flag
25         )
26
27         datapath.send_msg(mod) # Send the flow modification message to the switch

```

The routing strategy implemented in this SDN controller follows a **proactive** approach. Rather than reacting to packets arriving at the controller, flow rules are pre-installed on all switches during the controller initialization phase, specifically when the `switch_features_handler` is triggered upon OpenFlow handshake completion.

This proactive method offers several advantages:

- **Deterministic behavior:** All routing decisions are made statically before any traffic flows through the network, ensuring predictable and consistent packet forwarding;
- **Reduced latency:** Packets are forwarded according to pre-installed rules without requiring controller interaction, eliminating the overhead of reactive packet-in messages;
- **Improved performance:** The controller is not burdened with runtime packet processing, allowing it to manage multiple switches more efficiently;
- **Enhanced slice isolation:** Static rules guarantee that traffic is strictly confined within designated slices, preventing any cross-slice communication regardless of runtime conditions.

Unlike reactive approaches where the controller would process unknown flows dynamically, this proactive strategy ensures that every switch has complete knowledge of how to forward packets for all host pairs, thus achieving the isolation guarantees required by the network slicing architecture. When Mininet initiates a switch, it attempts to establish a TCP connection with the Ryu controller. If the connection is successful (i.e. the three-way handshake is completed), the OpenFlow handshake is then initiated. Subsequently, the switch sends a `FEATURES_REPLY` message, and the controller executes the `switch_features_handler` function. This function is responsible for installing the flow rules in the switch, which are defined based on the switch's unique identifier (`dpid`). The flow rules are designed to enforce the predefined routing paths and ensure that traffic is confined within the designated slices of the network.

```

1 @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2 def switch_features_handler(self, ev):
3     datapath = ev.msg.datapath
4     ofproto = datapath.ofproto
5     parser = datapath.ofproto_parser
6     dpid = datapath.id
7
8     if dpid == 1: # Switch S1
9         # UP flow configuration
10        port_out_Up = 3 # S1 out port (UP)
11        port_in_Up = 1 # S1 in port (UP)
12
13        # Flow IN
14        self.add_mac_flow(datapath, self.h1, self.h3, port_out_Up) # Eth H1 to H3 via s1:1
15        self.add_arp_flow(datapath, self.h1, port_out_Up) # ARP H1 to H3 via s1:1
16
17        # Flow Back
18        self.add_mac_flow(datapath, self.h3, self.h1, port_in_Up) # Eth H3 to H1 via s1:3
19        self.add_arp_flow(datapath, self.h3, port_in_Up) # ARP from H3 to H1 via s1:3
20
21        #####
22
23        # Down flow configuration
24        port_out_Dw = 4 # S1 out port (DW)
25        port_in_Dw = 2 # S1 in port (DW)

```

```

26
27     # Flow IN
28     self.add_mac_flow(datapath, self.h2, self.h4, port_out_Dw) # Eth H2 to H4 via s1:2
29     self.add_arp_flow(datapath, self.h2, port_out_Dw) # ARP H2 to H4 via s1:2
30
31     # Flow Back
32     self.add_mac_flow(datapath, self.h4, self.h2, port_in_Dw) # Eth H4 to H2 via s1:4
33     self.add_arp_flow(datapath, self.h4, port_in_Dw) # ARP H4 to H2 via s1:4
34
35     elif dpid == 2: # Switch S2
36         # UP flow configuration
37         port_out_Up = 1 # S2 out port (UP)
38         port_in_Up = 3 # S2 in port (UP)
39
40         # Flow IN
41         self.add_mac_flow(datapath, self.h3, self.h1, port_out_Up) # Eth H3 to H1 via s2:1
42         self.add_arp_flow(datapath, self.h3, port_out_Up) # ARP H3 to H1 via s2:1
43
44         # Flow Back
45         self.add_mac_flow(datapath, self.h1, self.h3, port_in_Up) # Eth H1 to H3 via s2:3
46         self.add_arp_flow(datapath, self.h1, port_in_Up) # ARP H1 to H3 via s2:3
47
48         #####
49
50         # Down flow configuration
51         port_out_Dw = 2 # S2 out port (DW)
52         port_in_Dw = 4 # S2 in port (DW)
53
54         # Flow IN
55         self.add_mac_flow(datapath, self.h4, self.h2, port_out_Dw) # Eth H4 to H2 via s2:2
56         self.add_arp_flow(datapath, self.h4, port_out_Dw) # ARP H4 to H2 via s2:2
57
58         # Flow Back
59         self.add_mac_flow(datapath, self.h2, self.h4, port_in_Dw) # Eth H2 to H4 via s2:4
60         self.add_arp_flow(datapath, self.h2, port_in_Dw) # ARP H2 to H4 via s2:4
61
62     elif dpid == 3: # Switch 3 (UP)
63         port_1 = 1 # S3 in port (UP)
64         port_2 = 2 # S3 out port (UP)
65
66         self.add_mac_flow(datapath, self.h1, self.h3, port_2) # Eth H1 to H3 via s3:2
67         self.add_arp_flow(datapath, self.h1, port_2) # ARP H1 to H3 via s3:2
68
69         self.add_mac_flow(datapath, self.h3, self.h1, port_1) # Eth H3 to H1 via s3:1
70         self.add_arp_flow(datapath, self.h3, port_1) # ARP H3 to H1 via s3:1
71
72
73     elif dpid == 4: # Switch 4 (DW)
74         port_1 = 1 # S4 in port (DW)
75         port_2 = 2 # S4 out port (DW)
76
77         self.add_mac_flow(datapath, self.h2, self.h4, port_2) # Eth H2 to H4 via s4:2
78         self.add_arp_flow(datapath, self.h2, port_2) # ARP H2 to H4 via s4:2
79
80         self.add_mac_flow(datapath, self.h4, self.h2, port_1) # Eth H4 to H2 via s4:1
81         self.add_arp_flow(datapath, self.h4, port_1) # ARP H4 to H2 via s4:1

```

After determining the switch identity with *dpid*, the controller creates the rule through `add_mac_flow`, which is the association $(\text{mac_src}, \text{mac_dst}) \rightarrow \text{port}$. Through the `add_arp_flow` rule, we enforce that ARP messages can exit on multiple ports, and it is a rule with higher priority than the one that enforces MAC-based flow. We observe that the ARP packet, which is sent in broadcast, is nevertheless confined within the network *slice*.

```

1     def add_mac_flow(self, datapath, src, dst, out_port, priority=10): # Utility function
2         # to add flow rules based on MAC addresses, specifying the source and destination
3         # MAC addresses, the output port, and an optional priority level for the flow rule
4         parser = datapath.ofproto_parser
5
6         match = parser.OFPMatch(eth_src=src, eth_dst=dst)
7         actions = [parser.OFPActionOutput(out_port)]
8         self.add_flow(datapath, priority, match, actions)

```

```

7
8 def add_arp_flow(self, datapath, eth_src, out_port): # Utility function to add flow
    rules for ARP packets, matching only the source MAC address and allowing the
    packet to be output on multiple ports (if out_port is a list), with a higher
    priority than the MAC-based flow rules to ensure that ARP packets are handled
    correctly
9 parser = datapath.ofproto_parser
10
11 match = parser.OFPMatch(eth_src=eth_src, eth_dst="ff:ff:ff:ff:ff:ff", eth_type=0
    x0806)
12 actions = []
13 if not isinstance(out_port, list):
14     out_port = [out_port]
15
16 for port in out_port:
17     actions.append(parser.OFPACTIONOutput(port))
18 self.add_flow(datapath, priority=20, match=match, actions=actions)

```

To test this configuration, Mininet's *pingall* tool and *Wireshark* were used.

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)

```

Figure 2: pingall results: hosts h1 and h3 can communicate, while hosts h2 and h4 can communicate, confirming the isolation of the two slices.

9	24.400751813	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.1
10	25.408132633	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.1
11	26.432216400	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.1
12	27.462619812	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.1
13	27.463452437	00:00:00_00:00:03	00:00:00_00:00:01	ARP	42 10.0.0.3 is at 00:00:00:00:00:03
14	27.464052836	10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request id=0x1d24, seq=1/256, ttl=64 (reply in 15)
15	27.464784378	10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply id=0x1d24, seq=1/256, ttl=64 (request in 14)
16	27.469362934	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.1
17	28.481793976	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.1
18	29.505254716	00:00:00_00:00:01	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.1
19	32.770710369	00:00:00_00:00:03	00:00:00_00:00:01	ARP	42 Who has 10.0.0.1? Tell 10.0.0.3
20	32.773166985	00:00:00_00:00:01	00:00:00_00:00:03	ARP	42 10.0.0.1 is at 00:00:00:00:00:01
21	36.750005723	10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) request id=0x010a, seq=1/256, ttl=64 (reply in 22)
22	36.750309553	10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) reply id=0x010a, seq=1/256, ttl=64 (request in 21)
23	36.755894448	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.3
24	37.761834881	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.3
25	38.784793016	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.2? Tell 10.0.0.3
26	39.812623014	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.3
27	40.832990270	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.3
28	41.857043208	00:00:00_00:00:03	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.3

Figure 3: Wireshark capture: Communication between h1 and h3 via the upper path (S3)

5	19.845205728	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.2
6	20.064514940	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.2
7	21.091256056	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.2
8	22.917892454	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.2
9	23.936498671	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.2
10	24.959805979	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.2
11	25.991624540	00:00:00_00:00:02	Broadcast	ARP	42 Who has 10.0.0.4? Tell 10.0.0.2
12	25.992709156	00:00:00_00:00:04	00:00:00_00:00:02	ARP	42 10.0.0.4 is at 00:00:00:00:00:04
13	25.993739998	10.0.0.2	10.0.0.4	ICMP	98 Echo (ping) request id=0x0682, seq=1/256, ttl=64 (reply in 14)
14	25.995187563	10.0.0.4	10.0.0.2	ICMP	98 Echo (ping) reply id=0x0682, seq=1/256, ttl=64 (request in 13)
15	31.232405450	00:00:00_00:00:04	00:00:00_00:00:02	ARP	42 Who has 10.0.0.2? Tell 10.0.0.4
16	31.233469894	00:00:00_00:00:02	00:00:00_00:00:04	ARP	42 10.0.0.2 is at 00:00:00:00:00:02
17	32.202639610	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.4
18	33.216380950	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.4
19	34.240378202	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.1? Tell 10.0.0.4
20	35.272094306	10.0.0.4	10.0.0.2	ICMP	98 Echo (ping) request id=0x1a8f, seq=1/256, ttl=64 (reply in 21)
21	35.272554579	10.0.0.2	10.0.0.4	ICMP	98 Echo (ping) reply id=0x1a8f, seq=1/256, ttl=64 (request in 20)
22	35.278429004	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.4
23	36.289502514	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.4
24	37.313437237	00:00:00_00:00:04	Broadcast	ARP	42 Who has 10.0.0.3? Tell 10.0.0.4

Figure 4: Wireshark capture: Communication between h2 and h4 via the lower path (S4)

2 Service slicing

In questa sezione verrà discusso il modo in cui è stato possibile effettuare lo *slicing* della rete al fine di differenziare il trattamento di diversi flussi. In questo caso particolare, sfruttando la stessa vista logica della rete utilizzata nella sezione precedente, facciamo in modo che lo *slice* inferiore sia destinato al traffico normale e alla connessione di tutti gli host tra di loro, mentre lo *slice* superiore sia destinato al traffico video. Per effettuare questa separazione, vengono implementati dei filtri sugli switch s1 ed s2. Gli switch s3 ed s4 controllano che effettivamente il flusso sia di tipo *streaming video*, altrimenti non inoltrano il pacchetto ed interrompono il flusso. Non sono state apportate modifiche alla struttura della rete tramite Mininet, quindi di seguito vengono esposte le modifiche alla classe del controllore Ryu.

```

1 @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
2 def switch_features_handler(self, ev):
3     datapath = ev.msg.datapath
4     ofproto = datapath.ofproto
5     parser = datapath.ofproto_parser
6     dpid = datapath.id
7     self.logger.info(f"[FEATURES HANDLER] dpid={dpid}")
8     if dpid == 1:
9         port_up, port_dw = 3, 4
10        port_h1, port_h2 = 1, 2
11        for host_port in [port_h1, port_h2]:
12            match = parser.OFPMatch(
13                in_port=host_port,
14                eth_type=0x0800,
15                ip_proto=17,
16                udp_dst=UDP_PORT_STREAMING
17            )
18            actions = [parser.OFPActionOutput(port_up)]
19            self.add_flow(datapath, priority=100, match=match, actions=actions)
20    elif dpid == 2:
21        port_up, port_dw = 1, 2
22        port_h3, port_h4 = 3, 4
23        for host_port in [port_h3, port_h4]:
24            match = parser.OFPMatch(
25                in_port=host_port,
26                eth_type=0x0800,
27                ip_proto=17,
28                udp_dst=UDP_PORT_STREAMING
29            )
30            actions = [parser.OFPActionOutput(port_up)]
31            self.add_flow(datapath, priority=100, match=match, actions=actions)
32    elif dpid == 3:
33        match = parser.OFPMatch(
34            in_port=1, eth_type=0x0800, ip_proto=17, udp_dst=UDP_PORT_STREAMING
35        )
36        actions = [parser.OFPActionOutput(2)]
37        self.add_flow(datapath, priority=100, match=match, actions=actions)
38        match = parser.OFPMatch(
39            in_port=2, eth_type=0x0800, ip_proto=17, udp_dst=UDP_PORT_STREAMING
40        )
41        actions = [parser.OFPActionOutput(1)]
42        self.add_flow(datapath, priority=100, match=match, actions=actions)
43    elif dpid == 4:
44        match = parser.OFPMatch(in_port=1)
45        actions = [parser.OFPActionOutput(2)]
46        self.add_flow(datapath, priority=1, match=match, actions=actions)
47        match = parser.OFPMatch(in_port=2)
48        actions = [parser.OFPActionOutput(1)]
49        self.add_flow(datapath, priority=1, match=match, actions=actions)
50    # regola di base -> manda pacchetto al controller
51    match = parser.OFPMatch()
52    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
53                                     ofproto.OFPCML_NO_BUFFER)]
54    self.add_flow(datapath, priority=0, match=match, actions=actions)

```

Il service based forwarding è palesato nelle istruzioni che riguardano il match dei pacchetti sugli switch marginali, in cui viene definita la regola per cui se il pacchetto usa il protocollo IP/UDP sul porto 9999 (streaming video) viene mandato sempre sul path superiore, indipendentemente dal MAC dei dispositivi comunicanti. Gli switch s3 ed s4 invece si occupano in questo caso solo del forwarding bidirezionale nel caso in cui avvenga il match (IP/UDP sul porto 9999 per lo switch s3 e tutti gli altri pacchetti per lo switch s4).

```

1 @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
2 def _packet_in_handler(self, ev):
3     msg = ev.msg
4     datapath = msg.datapath
5     ofproto = datapath.ofproto
6     parser = datapath.ofproto_parser
7     dpid = datapath.id
8     in_port = msg.match['in_port']
9
10    pkt = packet.Packet(msg.data)
11    eth = pkt.get_protocol(ethernet.ethernet)
12    if eth is None:
13        return
14    dst = eth.dst
15    src = eth.src
16    self.mac_to_port.setdefault(dpid, {})
17
18    self.mac_to_port[dpid][src] = in_port
19
20    # dest mac conosciuto
21    if dst in self.mac_to_port[dpid]:
22        out_port = self.mac_to_port[dpid][dst]
23        actions = [parser.OFPActionOutput(out_port)]
24    else:
25        host_ports = {
26            1: {1,2}, # [dpid] -> [h*,h*]
27            2: {3,4},
28            3: set(), # core UP
29            4: set() # core down
30        }
31
32        interswitch_links = {
33            1: {4}, # [dpid] -> down path
34            2: {2},
35        }
36
37        actions = []
38        for p in host_ports.get(dpid, set()):
39            if p != in_port:
40                actions.append(parser.OFPActionOutput(p))
41
42        for p in interswitch_links.get(dpid, set()):
43            if p != in_port:
44                actions.append(parser.OFPActionOutput(p))
45
46    if len(actions) == 1:
47        match = parser.OFPMatch(in_port=in_port, eth_src=src, eth_dst=dst)
48        self.add_flow(datapath, priority=1, match=match, actions=actions)
49
50    out = parser.OFPacketOut(
51        datapath=datapath,
52        buffer_id=msg.buffer_id,
53        in_port=in_port,
54        actions=actions,
55        data=msg.data
56    )
57    datapath.send_msg(out)

```

Questa funzione viene chiamata ogni volta che uno switch riceve un pacchetto, non trova nessun match e inoltra il pacchetto al controller. Il controller estrae informazioni dal pacchetto e mantiene una tabella MAC per ogni switch, dove si trova l'associazione (MAC, port). Se la destinazione è già nota, questa viene utilizzata per l'inoltro diretto. Se la destinazione è ignota, viene inviato il pacchetto in flooding solo sulla rotta down. Il test di questa implementazione è stato effettuato mediante i tool *pingall* di Mininet, *Wireshark* e *iPerf* per la simulazione di traffico.

Command Line

```
mininet> pingall
*** Ping: testing ping reachability

h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3

*** Results: 0% dropped (12/12 received)
```

La verifica con *iperf* e *wireshark* è stata effettuata configurando h1 come server e h4 come client. Lo scopo era evidenziare attraverso l'ascolto dei link collegati allo switch s3 che:

- le politiche di service slicing sono state effettivamente applicate, infatti il traffico "video streaming" è visto solo sul path superiore e tutto l'altro traffico non è mai visto sul path superiore;
- la banda della rete dedicata è maggiore di quella della rete *best effort* per costruzione, e quindi il traffico video gode di un throughput migliore.

Command Line

```
mininet> h1 iperf -s -u -p 9999
mininet> h4 iperf -c h1 -u -p 9999 -b 10M -t 10
mininet> h1 killall iperf
mininet> h1 iperf -s -u -p 5000
mininet> h4 iperf -c h1 -u -p 5000 -b 1M -t 10
```

[illegible]

10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request
10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply
10.0.0.1	10.0.0.3	ICMP	98 Echo (ping) request
10.0.0.3	10.0.0.1	ICMP	98 Echo (ping) reply
10.0.0.1	10.0.0.4	ICMP	98 Echo (ping) request
10.0.0.4	10.0.0.1	ICMP	98 Echo (ping) reply
10.0.0.2	10.0.0.3	ICMP	98 Echo (ping) request
10.0.0.1	10.0.0.4	ICMP	98 Echo (ping) request

[illegible]

